Parallel and Distributed Computing

Dr. Shajulin Benedict
shajulin@iiitkottayam.ac.in

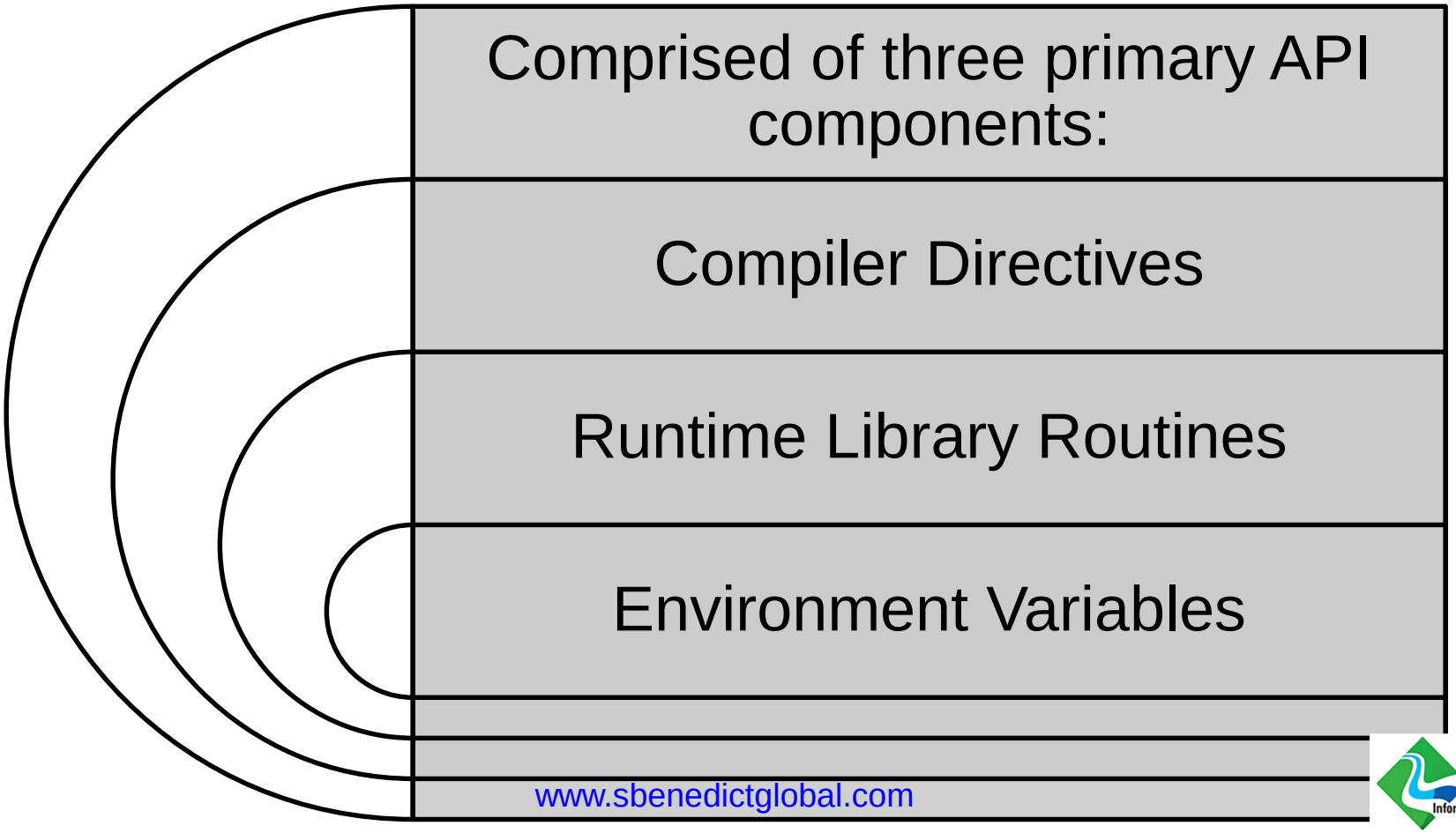
Indian Institute of Information Technology Kottayam

Syllabus

- Architectures – Multi-core and Many-core architectures, Accelerators (SIMD untis -Vectorization, GPUs), Goals of parallel systems.
-
- Applications – Scientific applications, Characteristics, requirements, regular grid applications, irregular applications, data dependence, parallelization process.
-
- Parallel Programming on Shared Memory – OpenMP (C++ languages), Execution Model, Shared and private data, Directives, Barriers, Sections, Run-Time library functions, scheduling strategies, Scalability study, OpenMP for accelerator programming.
-
- Parallel Programming on Distributed Memory – MPI, Collective operations, Non-Blocking, Collectives, Process topologies, Parallel I/O, Single sided communications.
-
- Performance Tools – Concepts, Event-Model execution, profiling, tracing, types of profiling, profiling tools – Scalasca, Score-P, MPI-P, EnergyAnalyzer, Tracing tools, Autotuning – Periscope Tuning Framework.

OpenMP – Open Multi-Processing

What is OpenMP? An Application Program Interface (API) that may be used to explicitly direct ***multi-threaded, shared memory*** parallelism.



What is OpenMP not?

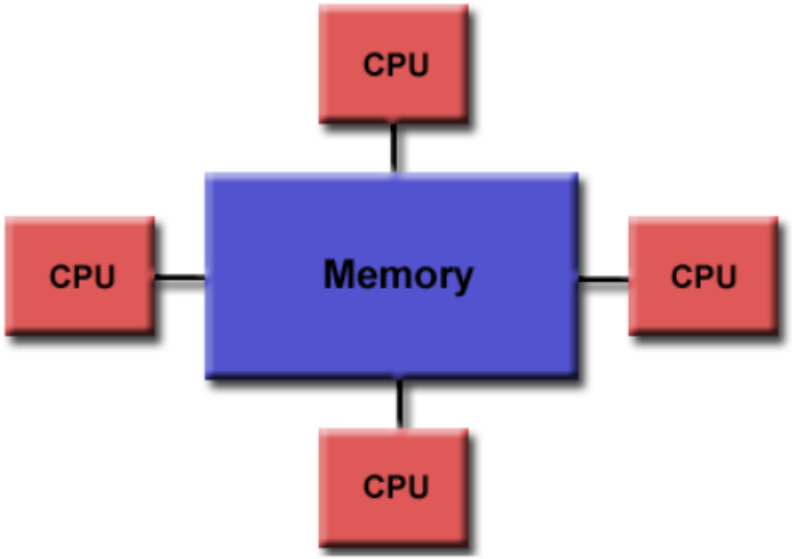
- It is not meant for distributed memory parallel systems (by itself).
- It is not necessarily implemented identically by all vendors.
- It is not guaranteed to make the most efficient use of shared memory.
- It is not required to check for data dependencies, data conflicts, race conditions, deadlocks, or code sequences that cause a program to be classified as non-conforming. It is the programmer's duty to do so.
- It is not designed to handle parallel I/O. The programmer is responsible for synchronizing input and output.

Release history

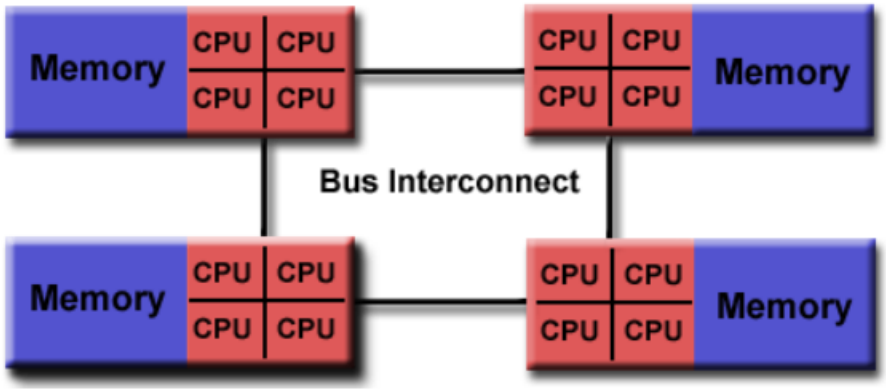
Date	Version
Oct 1997	Fortran 1.0
Oct 1998	C/C++ 1.0
Nov 1999	Fortran 1.1
Nov 2000	Fortran 2.0
Mar 2002	C/C++ 2.0
May 2005	OpenMP 2.5
May 2008	OpenMP 3.0
Jul 2011	OpenMP 3.1
Jul 2013	OpenMP 4.0
Nov 2015	OpenMP 4.5
Nov 2018	OpenMP 5.0
Nov 2024	OpenMP 6.0 (RELEASED…!

OpenMP programming Model

- It is designed for shared memory programming architectures. (Typically, single node)
- Both UMA (accessible primary memory) and NUMA (logical global address)



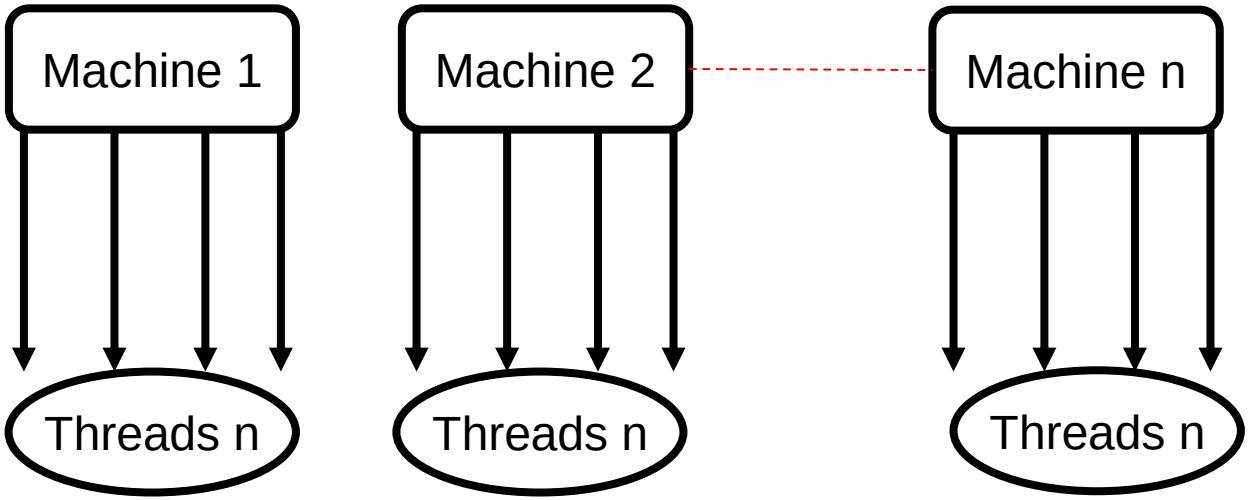
Uniform Memory Access



Non-Uniform Memory Access

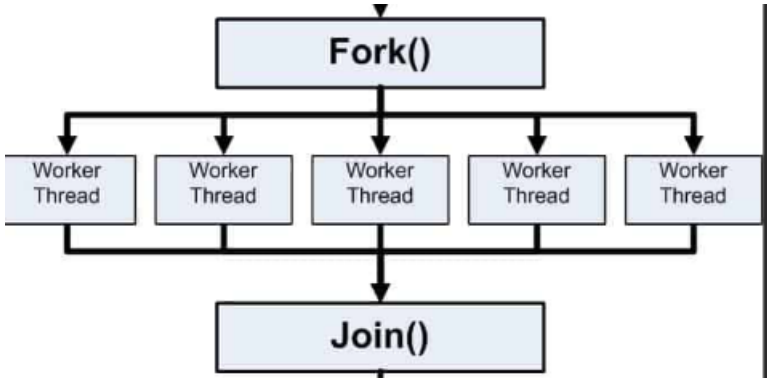
OpenMP in HPC

- OpenMP is applied in HPC with the combination of MPI (Hybrid MPI-OPenMP).



OpenMP

- Thread based programming model
  - i.e., it uses threads
  - Threads exist within the resources of a single process. Without the process, they cease to exist.
- It is an explicit programming model
  - i.e., programmer gets a full control over the parallelization.
  - Parallelization can be as simple as taking a serial program and inserting compiler directives....
- It is a Fork-Join execution model



Fork-Join Model

- All OpenMP programs begin as a single process: the **master thread**. The master thread executes sequentially until the first **parallel region** construct is encountered.
- **FORK:** the master thread then creates a team of parallel ***threads***.
- JOIN: After completion, the team of parallel threads join to the master thread.

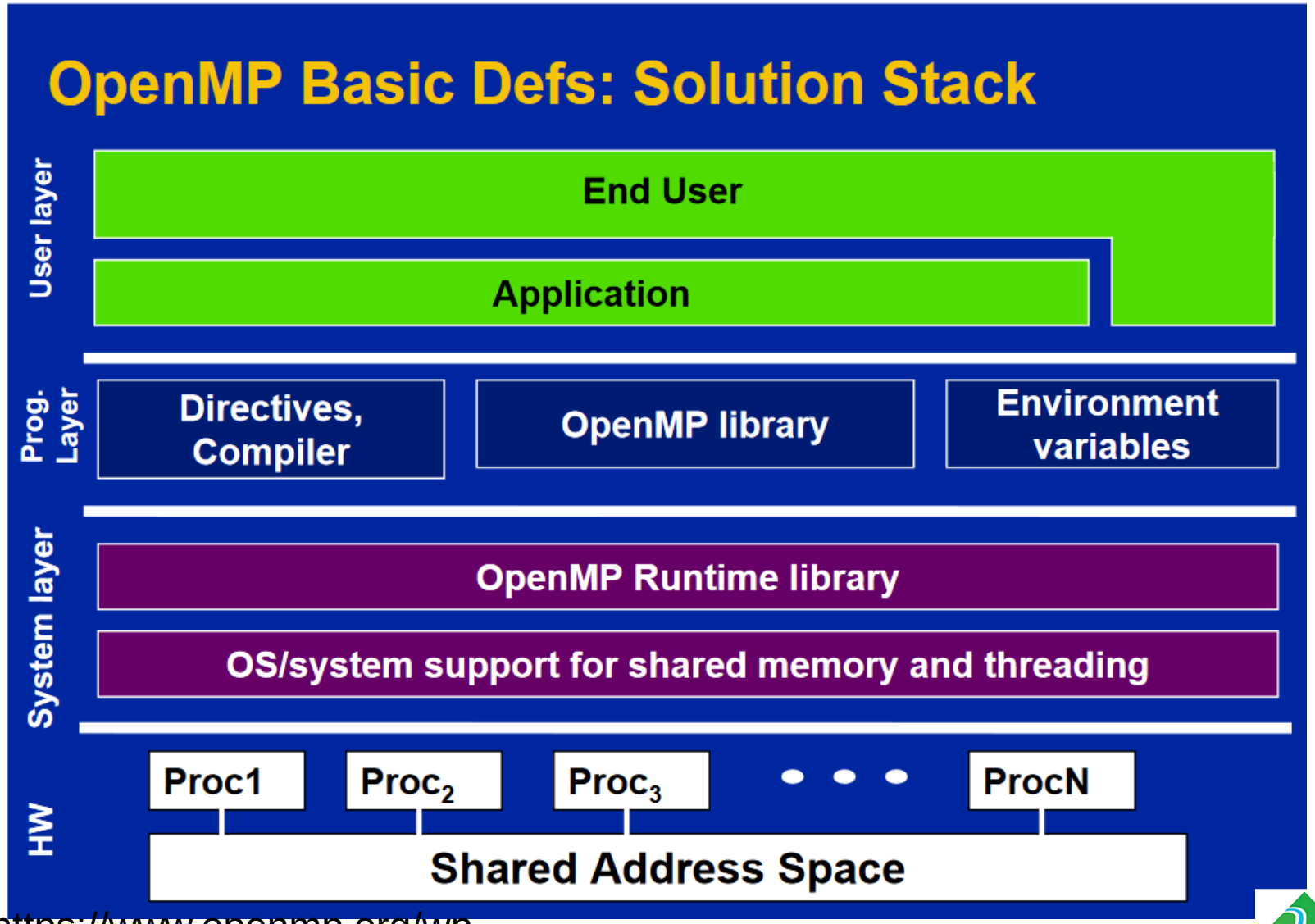
Fork-Join Model

- Because OpenMP is a shared memory programming model, most data within a parallel region is shared by default.
- All threads in a parallel region can access this shared data simultaneously.

- Nested parallelism is possible
  - Fork within a fork…
  - parallel regions inside other parallel regions

- Dynamic threads are possible
  - The API provides for the runtime environment to dynamically alter the number of threads used to execute parallel regions

OpenMP solution stack



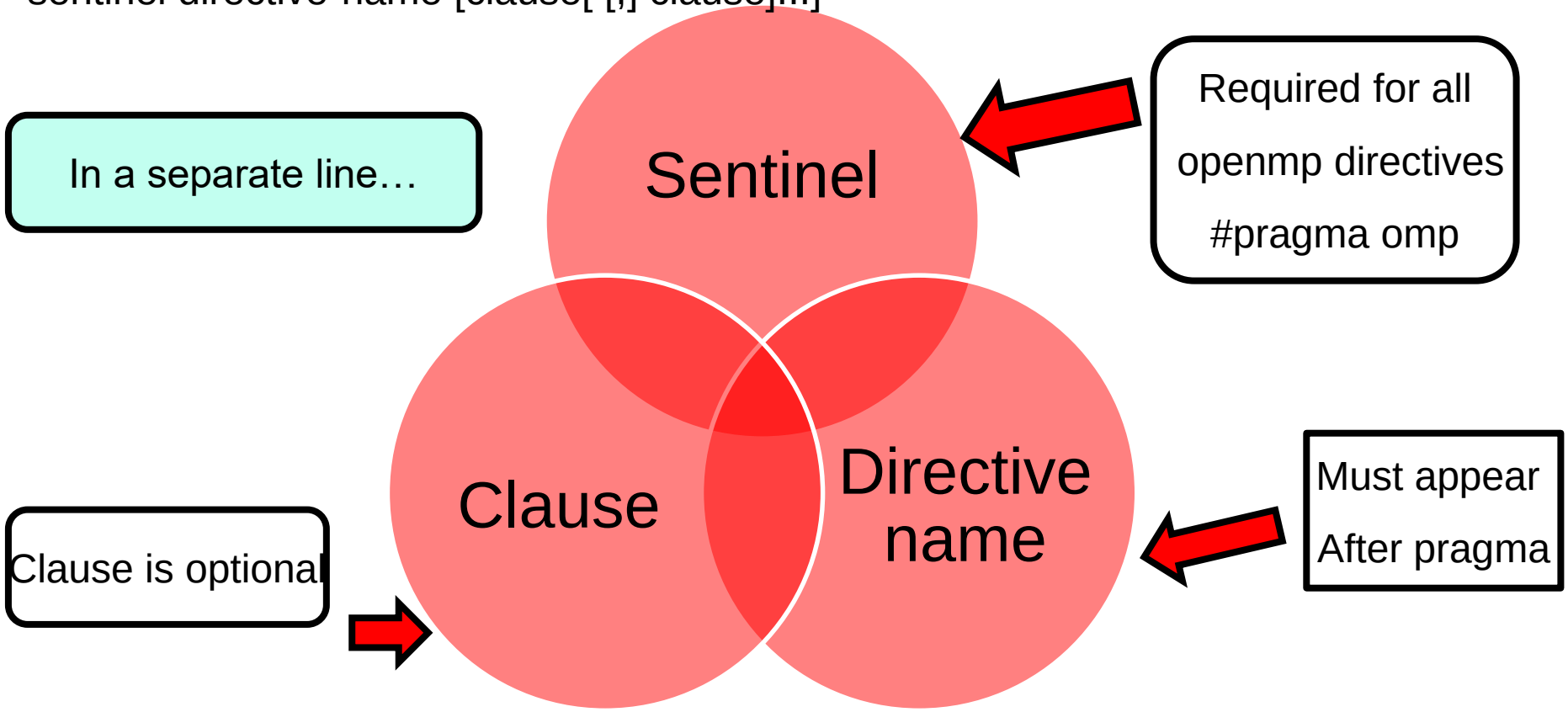
https://www.openmp.org/wp-content/uploads/omp-hands-on-SC08.pdf

Compiler Directives

- Compiler directives appear as comments in your source code and are ignored by compilers unless you tell them otherwise - usually by specifying the appropriate compiler flag.
- OpenMP compiler directives are used for various purposes:
  - Spawning a parallel region
  - Dividing blocks of code among threads
  - Distributing loop iterations between threads
  - Serializing sections of code
  - Synchronization of work among threads

Compiler Directive Syntax

sentinel directive-name [clause[[,] clause]...]



Fortran	`!$OMP PARALLEL DEFAULT(SHARED) PRIVATE(BETA,PI)`
C/C++	`#pragma omp parallel default(shared) private(beta,pi)`

OpenMP code strucuture

```
#include <omp.h>

main ()  {

int var1, var2, var3;

Serial code
      .
      .
      .

Beginning of parallel region. Fork a team of threads.
Specify variable scoping

#pragma omp parallel private(var1, var2) shared(var3)
   {

   Parallel region executed by all threads
            .
   Other OpenMP directives
            .
   Run-time Library calls
            .
   All threads join master thread and disband

   }

Resume serial code
      .
      .
      .

}
```

OpenMP compiler flags

Compiler / Platform	Compiler Commands	OpenMP Flag

Intel
Linux
	`icc`
	`icpc`
	`ifort`
		`-qopenmp`

GNU
Linux
IBM Blue Gene
Sierra, CORAL EA
	`gcc`
	`g++`
	`g77`
	`gfortran`
		`-fopenmp`

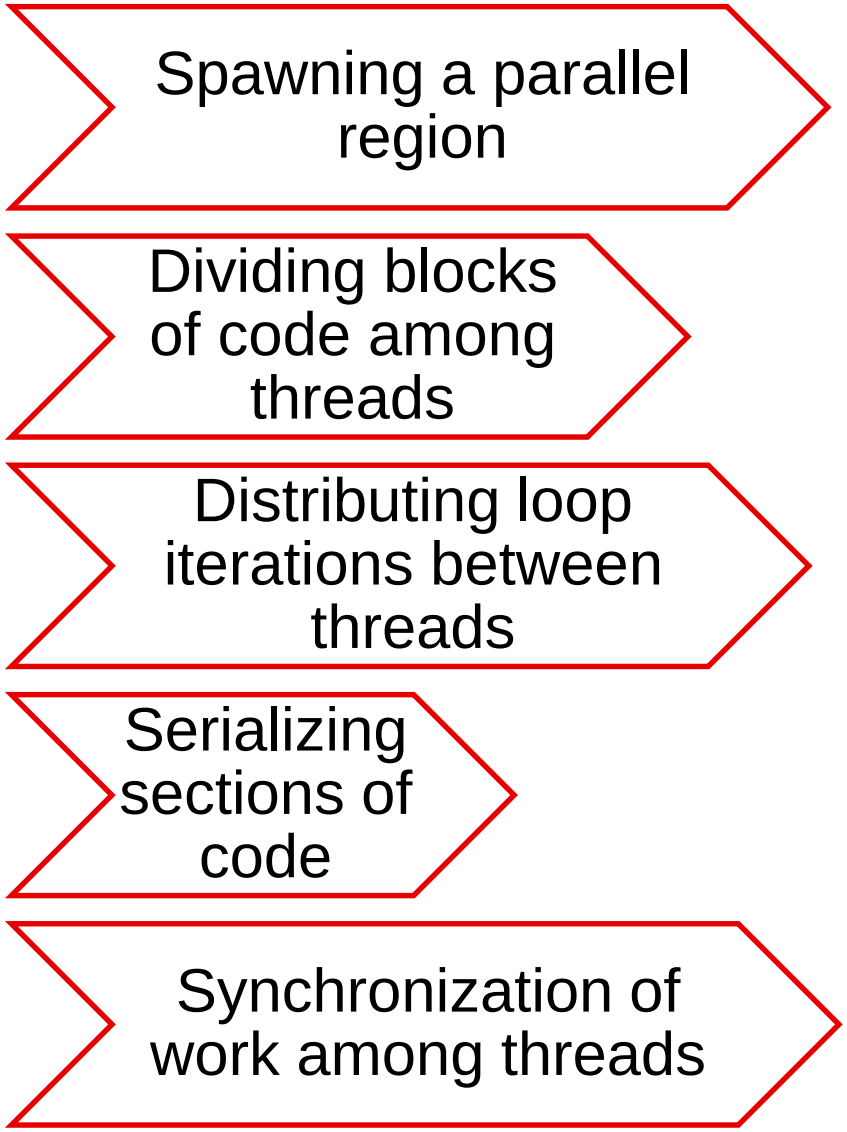
PGI
Linux
Sierra, CORAL EA
	`pgcc`
	`pgCC`
	`pgf77`
	`pgf90`
		`-mp`

Clang Linux Sierra, CORAL EA	`clang` `clang++`	`-fopenmp`

IBM XL
Blue Gene
	`bgxlc_r, bgcc_r`
	`bgxlC_r, bgxlc++_r`
	`bgxlc89_r`
	`bgxlc99_r`
	`bgxlf_r`
	`bgxlf90_r`
	`bgxlf95_r`
	`bgxlf2003_r`
		`-qsmp=omp`

IBM XL
Sierra, CORAL EA
	`xlc_r`
	`xlC_r, xlc++_r`
	`xlf_r`
	`xlf90_r`
	`xlf95_r`
	`xlf2003_r`
	`xlf2008_r`
		`-qsmp=omp`

OpenMP Directives – purposes (repeated)



OpenMP Constructs – Spawning Parallel Region

#pragma omp parallel *[clause ...] newline*

Clauses are …

if *(scalar_expression)*
private *(list)*
shared *(list)*
default (shared | none)
firstprivate *(list)*
reduction *(operator: list)*
copyin *(list)*
num_threads *(integer-expression)*

OpenMP constructs – Parallel Region

•When a thread reaches a PARALLEL directive, it creates a team of threads and becomes the master of the team.
•The master is a member of that team and has thread number 0 within that team.
•Starting from the beginning of this parallel region, the code is duplicated and all threads will execute that code.
•There is an implied barrier at the end of a parallel region. Only the master thread continues execution past this point.
•If any thread terminates within a parallel region, all threads in the team will terminate, and the work done up until that point is undefined.

Number of threads in a Parallel Region

- The number of threads in a parallel region is determined by the following factors, in order of precedence:
  1. Evaluation of the **IF** clause
  2. Setting of the **NUM_THREADS** clause
  3. Use of the **omp_set_num_threads()** library function
  4. Setting of the **OMP_NUM_THREADS** environment variable
  5. Implementation default - usually the number of CPUs on a node, though it could be dynamic.
- Threads are numbered from 0 (master thread) to N-1

OpenMP C – Hello world program

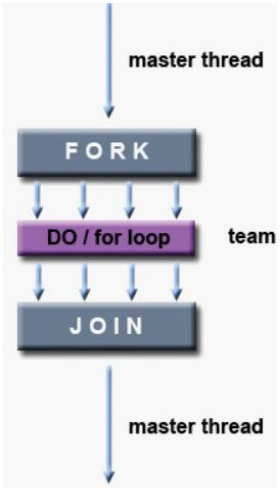
```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char *argv[])
{
int nthreads, tid;

/* Fork a team of threads giving them their own copies of variables */
#pragma omp parallel private(nthreads, tid)
  {

  /* Obtain thread number */
  tid = omp_get_thread_num();
  printf("Hello World from thread = %d\n", tid);

  /* Only master thread does this */
  if (tid == 0)
    {
    nthreads = omp_get_num_threads();
    printf("Number of threads = %d\n", nthreads);
    }

  }  /* All threads join master thread and disband */

}
```

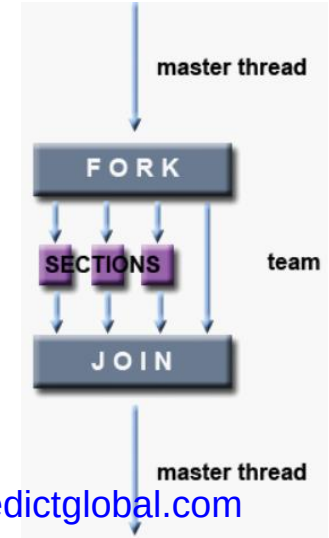
OpenMP construct – Work Sharing

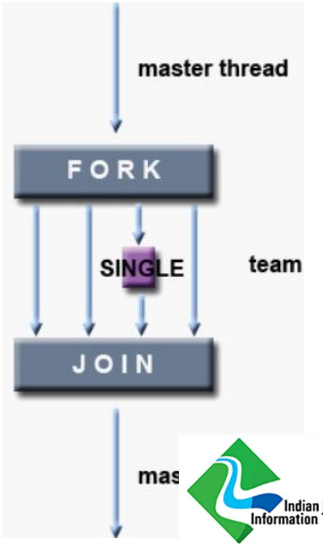
- A work-sharing construct divides the execution of the enclosed code region among the members of the team that encounter it.
- It assumes that the section of code runs within a parallel region.
- Work-sharing constructs do not launch new threads.
- There is no implied barrier upon entry to a work-sharing construct, however there is an implied barrier at the end of a work sharing construct.

Types of Worksharing construct

- DO/FOR
  - shares iterations of a loop across the team. Represents a type of "data parallelism”
- SECTIONS
  - breaks work into separate, discrete sections. Each section is executed by a thread. Can be used to implement a type of "functional parallelism“.
- SINGLE
  - serializes a section of code







Worksharing – OMP for

```
#include <omp.h>
#define N 1000
#define CHUNKSIZE 100

main(int argc, char *argv[]) {

int i, chunk;
float a[N], b[N], c[N];

/* Some initializations */
for (i=0; i < N; i++)
  a[i] = b[i] = i * 1.0;
chunk = CHUNKSIZE;

#pragma omp parallel shared(a,b,c,chunk) private(i)
  {

  #pragma omp for schedule(dynamic,chunk) nowait
  for (i=0; i < N; i++)
    c[i] = a[i] + b[i];

  }   /* end of parallel region */

}
```

The **NOWAIT** clause allows thread to continue execution of a parallel region without waiting for the other threads in the team to complet region.

Worksharing – OMP Section

```
#include <omp.h>
#define N 1000

main(int argc, char *argv[]) {

int i;
float a[N], b[N], c[N], d[N];

/* Some initializations */
for (i=0; i < N; i++) {
  a[i] = i * 1.5;
  b[i] = i + 22.35;
  }

#pragma omp parallel shared(a,b,c,d) private(i)
  {

  #pragma omp sections nowait
    {

    #pragma omp section
    for (i=0; i < N; i++)
      c[i] = a[i] + b[i];

    #pragma omp section
    for (i=0; i < N; i++)
      d[i] = a[i] * b[i];

    }  /* end of sections */

  }  /* end of parallel region */

}
```

OpenMP directives for Master/Synchronization

Master	Specifies that only the master thread should execute a section of the program.
Critical	Specifies that code is only executed on one thread at a time.
Barrier	Synchronizes all threads in a team; all threads pause at the barrier, until all threads execute the barrier.
Atomic	Specifies that a memory location that will be updated atomically.
Flush	Specifies that all threads have the same view of memory for all shared objects.
ordered	Specifies that the code in a loop should be executed like a sequential loop.

https://docs.microsoft.com/en-us/cpp/parallel/openmp/reference/openmp-directives?view=vs-2019

OpenMP -- Atomic

- Specifies that a memory location that will be updated atomically.

```
// omp_atomic.cpp
// compile with: /openmp
#include <stdio.h>
#include <omp.h>

#define MAX 10

int main() {
   int count = 0;
   #pragma omp parallel num_threads(MAX)
   {
      #pragma omp atomic
      count++;
   }
   printf_s("Number of threads: %d\n", count);
}
```

Output

```
Number of threads: 10
```

OpenMP - Barrier

- Synchronizes all threads in a team; all threads pause at the barrier, until all threads execute the barrier.
- Syntax
  - #pragma omp barrier
- Applied within a parallel region of OpenMP

OpenMP - Master

- Specifies that only the master thread should execute a section of the program.

```
// compile with: /openmp
#include <omp.h>
#include <stdio.h>

int main( )
{
    int a[5], i;

    #pragma omp parallel
    {
        // Perform some computation.
        #pragma omp for
        for (i = 0; i < 5; i++)
            a[i] = i * i;

        // Print intermediate results.
        #pragma omp master
            for (i = 0; i < 5; i++)
                printf_s("a[%d] = %d\n", i, a[i]);

        // Wait.
        #pragma omp barrier

        // Continue with the computation.
        #pragma omp for
        for (i = 0; i < 5; i++)
            a[i] += i;
    }
}
```

Output

```
a[0] = 0
a[1] = 1
a[2] = 4
a[3] = 9
a[4] = 16
```

OpenMP - Ordered

- Specified that a code under a parallelized *for* loop should be executed like a sequential loop.
- #pragma omp ordered

  structured-block

OpenMP - Single

- Lets you specify that a section of code should be executed on a single thread, not necessarily the master thread.
- There is an implicit barrier for Single and For…
- But, there is no implicit barrier for master.

C++

```
// omp_single.cpp
// compile with: /openmp
#include <stdio.h>
#include <omp.h>

int main() {
   #pragma omp parallel num_threads(2)
   {
      #pragma omp single
      // Only a single thread can read the input.
      printf_s("read input\n");

      // Multiple threads in the team compute the results.
      printf_s("compute results\n");

      #pragma omp single
      // Only a single thread can write the output.
      printf_s("write output\n");
    }
}
```

Output

```
read input
compute results
compute results
write output
```

OpenMP Runtime library Routines

omp_set_num_threads(n)	Sets the number of threads that will be used in the next parallel region
omp_get_num_threads()	Returns the number of threads that are currently in the team executing the parallel region from which it is called
omp_get_max_threads()	Returns the maximum value that can be returned by a call to the OMP_GET_NUM_THREADS function
omp_get_thread_num()	Returns the thread number of the thread, within the team, making this call.
omp_get_thread_limit()	Returns the maximum number of OpenMP threads available to a program
omp_get_num_procs()	Returns the number of processors that are available to the program

https://computing.llnl.gov/tutorials/openMP/#Directives

OpenMP Runtime library Routines

omp_in_parallel	Used to determine if the section of code which is executing is parallel or not
omp_set_dynamic	Enables or disables dynamic adjustment (by the run time system) of the number of threads available for execution of parallel regions
omp_get_dynamic	Used to determine if dynamic thread adjustment is enabled or not
omp_set_nested	Used to enable or disable nested parallelism
omp_get_nested	Used to determine if nested parallelism is enabled or not
omp_set_schedule	Sets the loop scheduling policy when "runtime" is used as the schedule kind in the OpenMP directive
omp_get_schedule	Returns the loop scheduling policy when "runtime" is used as the schedule kind in the OpenMP directive
omp_set_max_active_levels	Sets the maximum number of nested parallel regions

OpenMP runtime library routines

- For C/C++, all of the run-time library routines are actual subroutines.

Fortran	`INTEGER FUNCTION OMP_GET_NUM_THREADS()`
C/C++	`#include <omp.h>` `int omp_get_num_threads(void)`

OpenMP Tasks

- OpenMP Task feature is available starting with OpenMP3.0.
- A task is composed of
  - Code to be executed
  - Data environment (inputs to be used and outputs to be generated)
  - A location where the task will be executed (which thread?).
- In the previous context…
  - The tasks were initially implicit in OpenMP.
  - A parallel directive constructs implicit tasks, one per thread.
  - They synchronize with the master thread using a barrier once all parallel tasks are complete.
- In OpenMP tasks, the tasks enable dynamic parallelism (also, fine-grained parallelism)

OpenMP Tasks

- We could have different OpenMP tasks per parallel construct per thread…
- OpenMP Task Concepts….
  - When a thread encounters a task construct, it may choose to execute the task immediately or defer its execution until a later time.
  - If deferred, the task is placed in a conceptual pool of tasks associated with the current parallel region.
  - All team threads will take tasks out of the pool and execute them until the pool is empty.
  - A thread that executes a task might be different from the thread that originally encountered it.
- Compared to OpenMP for loops,
  - Tasks could enabled dynamism in working with tasks.

OpenMP Tasks -- Advantages

- It has an elastic behavior – i.e., the threads have a choice between multiple existing tasks.
- Few scheduling strategies are implemented (efficient)
- It has a different perspective than the traditional fork/join model of OpenMP constructs.
- The code associated with a task construct will be executed only once.
- 2 ways how OpenMP tasks are associated with parallel threads.
  - Tied
    - A task is tied if the code is executed by the same thread from beginning to end.
  - Untied
    - Otherwise, the task is untied (the code can be executed by more than one thread).

OpenMP Tasks example

- Similar to the worksharing construct.
- But, tasks have more flexibility in the execution orders.

```
#pragma omp parallel
{
#pragma omp task
    printf("hello world from a random thread\n");
}
```

- Typically, within a parallel → single region

```
#pragma omp parallel
{
#pragma omp single
{
#pragma omp task
    printf("hello world\n");

#pragma omp task
    printf("hello again!\n");
}}
```

https://www.nersc.gov/users/software/programming-models/openmp/openmp-tasking/openmp-task-intro/

OpenMP tasks

- In the previous example, the order of execution of OpenMP tasks is not guaranteed.
- i.e., Hello again might be printed at first.
- If you need to guarantee the ordering within tasks, you have to include taskwait.

```
#pragma omp parallel
{
#pragma omp single
{
#pragma omp task
    printf("hello world\n");

#pragma omp taskwait

#pragma omp task
    printf("hello again!\n");
}}
```

OpenMP tasks

```
#include <stdio.h>
int main() {
    #pragma omp parallel
    {
        int number = 1;
        #pragma omp task
        {
            printf("I think the number is %d.\n", number);
            number++;
        }
    }
    return 0;
}
```

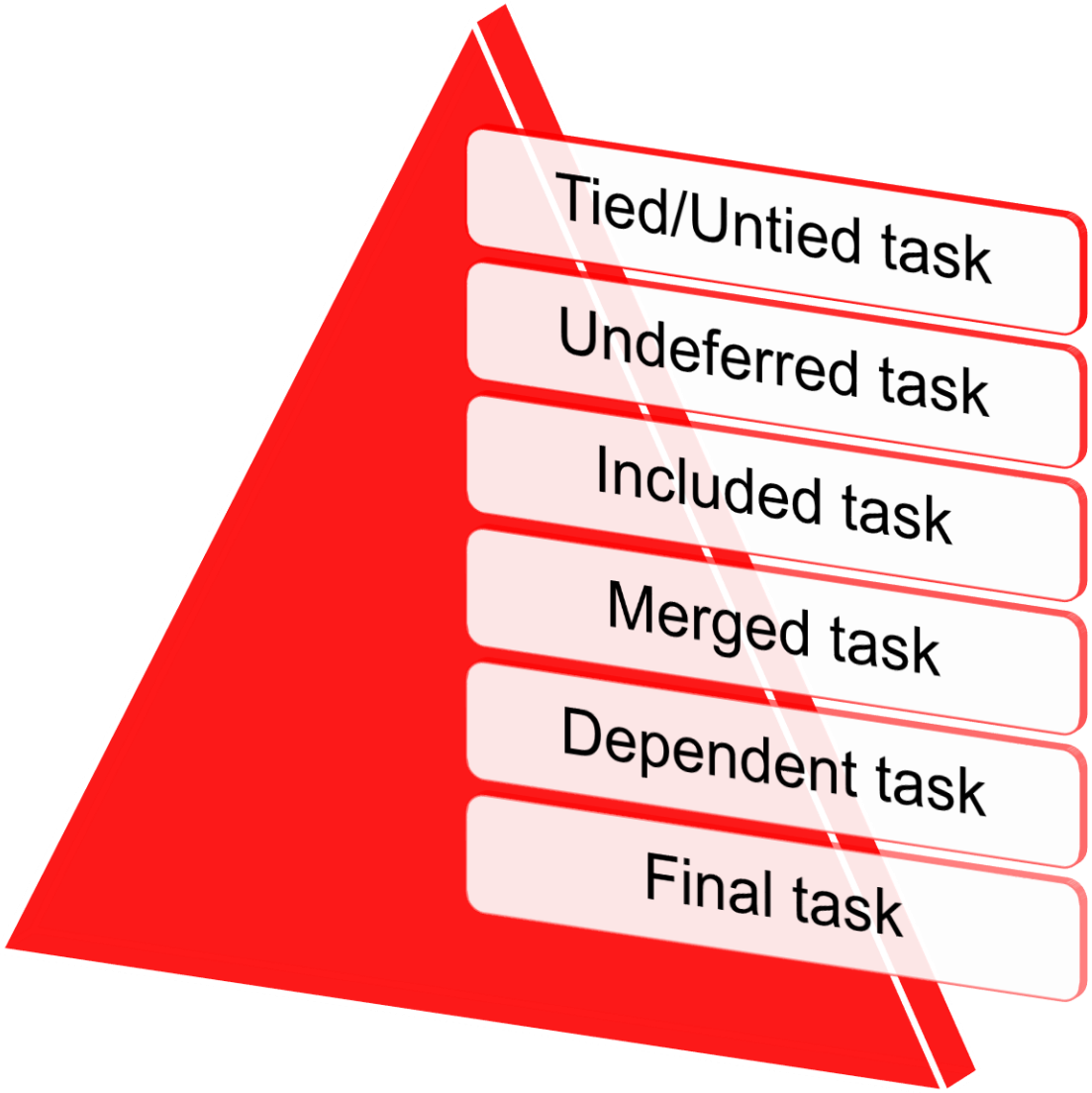
```
$ gcc -o task-ex task.c –
fopenmp
$ ./task-ex
I think the number is 1.
I think the number is 1.
I think the number is 1.
I think the number is 1.
```

OpenMP tasks

```
#include <stdio.h>
int main() {
    #pragma omp parallel
     #pragma omp single nowait
    {
        int number = 1;
        #pragma omp task
        {
            printf("I think the number is %d.\n", number);
            number++;
        }
    }
    return 0;
```

```
$ gcc -o task-ex task.c –
fopenmp
$ ./task-ex
I think the number is 1.
```

Types of OpenMP tasks



Types of OpenMP tasks

- Undeferred OpenMP task
  - Deferred Meaning – Postposed or Delayed or Suspended
  - Task creations are not deferred (task creations are not delayed).
  - i.e., tasks are created and moved to the conceptual pool for future executions. Hence, the undeferred tasks might not be executed immediately by the encountering thread.
  - The task might be generated by one thread and got executed by some other thread.
- Included OpenMP Task
  - an **included** task is executed immediately by the encountering thread and is not placed in the pool to be executed at a later time.
  - the generation of the tasks is suspended until the execution of the included task is completed.
  - Only after the execution step, the generating task can resum

Merged OpenMP Tasks

- A **merged** task is a task whose data environment is the same as that of its generating task region.
- If a merged task is generated, the behavior is equivalent to the normal execution of a parallel region.

Dependent OpenMP task

- Tasks whose execution order is constrained by dependencies on other tasks.
- They are specified using the depend clause (e.g., in, out, inout).
- inout indicates read-write dependency.

```
#include <stdio.h>
#include <omp.h>

int main() {
    int a = 5, b = 10, c = 0;

    #pragma omp parallel
    #pragma omp single
    {
        // Task 1: Compute a + b → c (in: a,b; out: c)
        #pragma omp task depend(in: a, b) depend(out: c)
        {
            c = a + b;
            printf("Task 1 computed c = %d (a + b)\n", c);
        }
        // Task 2: Use c to compute a new value (in: c; out: a)
        #pragma omp task depend(in: c) depend(out: a)
        {
            a = c * 2;
            printf("Task 2 updated a = %d (c * 2)\n", a);
        }
        // Task 3: Print final a and c (inout: a,c)
        #pragma omp task depend(inout: a, c)
        {
            printf("Task 3 final values -> a = %d, c = %d\n", a, c);
        }
    }
    return 0;
}
```

Final OpenMP task

- A final task is a task that forces all of its descendent tasks to become final (if encountered).
- It is written as a clause in the task directive (along with an expression).

OpenMP Task Clauses

- Clause can be
  - depend (list)
  - if (expression)
  - final (expression)
  - untied
  - mergeable
  - default (shared | firstprivate | none)
  - private (list)
  - firstprivate (list)
  - shared (list)
  - priority (value)

OpenMP nested parallelism

- OpenMP has support for nested parallelism
- Per-thread internal control variables
  - Allows, for example, calling omp_set_num_threads() inside a parallel region.
  - Controls the team sizes for next level of parallelism
- Library routines to determine depth of nesting, IDs of parent/grandparent etc. threads, team sizes of parent/grandparent etc.
  - omp_get_active_level()
  - omp_get_ancestor(level)
  - omp_get_teamsize(level)

OpenMP Taskwait

- The taskwait construct specifies a wait on the completion of child tasks of the current task.
- The taskwait construct is a stand-alone directive.

- Syntax:

#pragma omp taskwait [clause[[,] clause] ...] new-line

OpenMP Task example

```
#include <stdio.h>
#include <omp.h>

int main(int argc, char **argv)
{
   int i;
   int sum = 0;

   #pragma omp parallel shared(sum)
   {
      #pragma omp single
      {
         #pragma omp task
         {
            for (i = 0; i < 1000; i++) {
               a[i] = i;
               sum = sum + a[i];
            }
         }

         // Need "#pragma omp taskwait" here

         printf("%d\n", sum); // sum is indeterminate here
      }
   }
   return 0;
}
```

Fibonacci Example – OpenMP Tasks

```
#include <stdio.h>
#include <omp.h>
#define THRESHOLD 9
        int fib(int n)
        {
          int i, j;
          if (n<2)
            return n;
          #pragma omp task shared(i) firstprivate(n) final(n <= THRESHOLD)
          i=fib(n-1);
          #pragma omp task shared(j) firstprivate(n) final(n <= THRESHOLD)
          j=fib(n-2);
          #pragma omp taskwait
          return i+j;
        }
 int main()
{
  int n = 30;
  omp_set_num_threads(4);
  #pragma omp parallel shared(n)
  {
     #pragma omp single
     printf ("fib(%d) = %d\n", n, fib(n));
  }
}
```

OpenMP Taskloop

- The taskloop construct specifies that the iterations of one or more associated loops will be executed in parallel using explicit tasks.
- It was introduced in OpenMP 4.5.
- The iterations are distributed across tasks generated by the construct and scheduled to be executed.

```
#pragma omp taskloop [clause[[,] clause] ...] new-line
    for-loops
```

OpenMP TaskGroup

```
#include <stdio.h>

int main() {
    #pragma omp parallel
    #pragma omp single nowait
    {
        #pragma omp taskgroup
        {
            #pragma omp task
            {
                #pragma omp task
                printf("Hello.\n");

                printf("Hi.\n");
            }
        }
        printf("Goodbye.\n");
    }
    return 0;
}
```

A taskgroup defines a scope in which multiple tasks can be created.

Only after the completion of all tasks in the taskgroup, the newer tasks will be executed.

Answer to the code:

Hi.

Hello.

Goodbye.

Rerun:

Hello.

Hi.

Goodbye.

OpenMP Collapse of Nested Parallelism

- Use the OpenMP collapse-clause to increase the total number of iterations that will be partitioned across the available number of OMP threads by reducing the granularity of work …
- i.e., Specifies how many loops in a nested loop should be collapsed into one large iteration space and divided according to the schedule clause.
- If the amount of work to be done by each thread is non-trivial (after collapsing is applied), this may improve the parallel scalability of the OMP application done by each thread.
-

```
#pragma omp parallel for collapse(2)
  for (i = 0; i < imax; i++) {
    for (j = 0; j < jmax; j++) a[ j + jmax*i] = 1.;
  }
```

Scheduling strategies – clauses (OpenMP FOR)

Schedule Clause	When To Use
STATIC	Pre-determined and predictable by the programmer
DYNAMIC	Unpredictable, highly variable work per iteration
GUIDED	Special case of dynamic to reduce scheduling overhead

Least work at runtime : scheduling done at compile-time

Most work at runtime : complex scheduling logic used at run-time

Quiz

- https://cvw.cac.cornell.edu/OpenMP/quiz

- Further reading

Training OpenMP
Language: English
Access: free (online exercises and downloads)
Available at: http://www.hpc.cineca.it/content/training-openmp

Material of OpenMP training
Language: English
Access: free download
Available at: http://www.hpc.cineca.it/sites/default/files/OpenMP.zip

OpenMP 6.0

- https://www.openmp.org/